

Thapar Institute of Engineering and Technology

Project Proposal for UCS503P

AetherGrid AI

An Autonomous Logistics and Emergency-Dispatch Simulation

*A simulation-driven intelligent dispatch system for
dynamic routing, optimal vehicle assignment, and predictive
idle-vehicle repositioning*

Presented by: Vishal Singla

Roll No.: 1024240009

Presented by: Sparsh Verma

Roll No.: 1024240011

Course: Software Engineering(UCS503P)

Presented to: Dr. Zeelani

August 2026

1 Abstract

AetherGrid AI is a simulation-based intelligent dispatch system designed to study how a mixed fleet of emergency and logistics vehicles can be managed in a changing urban environment. The system models a city as a dynamic road network in which congestion, weather, hazards, and road closures can continuously change the cost of travel. It then combines pathfinding, vehicle-request assignment, multi-stop route optimization, and reinforcement learning to make dispatch decisions.

The project is designed as a seeded and reproducible discrete-event simulation. Emergency and logistics requests arrive continuously, are collected in short dispatch batches, and are assigned to suitable vehicles. Dynamic A* is used for route execution, while Dijkstra is used to calculate the travel-time matrix required by the assignment stage. The Hungarian algorithm is used for one-to-one batch assignment, while a Genetic Algorithm is used for multi-drop logistics routes with time windows. A Q-Learning or Deep Q-Network (DQN) based policy is used only for repositioning idle vehicles between dispatch batches. This separation ensures that the learning component cannot interrupt or invalidate an active emergency dispatch.

A real-time Mission Control dashboard will display vehicles, requests, routes, road conditions, telemetry, and simulation statistics. The project will be evaluated against simple baseline strategies using response time, service-level compliance, route cost, energy consumption, runtime, and reproducibility. The primary objective is not to claim a new algorithm, but to demonstrate the correct and measurable integration of established algorithms into a complete real-time simulation system.

2 Introduction

Urban emergency response and logistics operations require vehicles to be assigned to requests quickly and intelligently. The difficulty is not only finding a short route. The system must also decide which vehicle should serve which request, account for changing road conditions, handle several requests at the same time, and maintain good vehicle positions for future demand.

A conventional nearest-vehicle strategy can work well in simple situations, but it becomes less effective when demand is uneven, roads become unavailable, or several vehicles compete for the same requests. AetherGrid addresses this problem through a simulation environment in which the complete dispatch process can be observed, tested, and compared under controlled conditions.

The immediate engineering objective is to build a reproducible simulation engine that combines a live cost-priced street graph with three cooperating decision layers:

1. dynamic pathfinding,
2. batch-optimal vehicle assignment, and
3. learned repositioning of idle vehicles.

These layers are exposed through a real-time Mission Control dashboard so that the behavior of the system can be observed during live scenarios.

3 Problem Statement

Dispatching a heterogeneous fleet such as ambulances, fire engines, drones, and delivery vans against a continuous stream of requests is a difficult optimization problem. Streets, demand, vehicle availability, and operating conditions can all change while the system is running.

The major problems considered in AetherGrid are described below.

3.1 Combinatorial Assignment Problem

If several vehicles and several requests are available at the same time, there may be many possible vehicle-request combinations. Trying every possible combination is inefficient for real-time dispatch.

A system therefore needs an assignment strategy that can find a high-quality solution without testing every possible arrangement.

3.2 Changing Road Conditions

The shortest route under normal conditions may not remain the best route when congestion, weather, hazards, or road closures occur.

Therefore, the road network must be treated as dynamic rather than static.

3.3 Myopic Dispatch

A nearest-vehicle strategy considers the current request but may ignore the effect of its decision on future requests. Assigning the only nearby vehicle to a current request may leave an important area without a suitable vehicle.

AetherGrid therefore separates immediate dispatch from future positioning.

3.4 Disruption Handling

The system must continue to operate when roads are closed or hazards are introduced. A closed road must never be selected by a route.

This requirement is important because the project is intended to demonstrate not only normal-case routing but also graceful behavior under disruption.

4 Aim and Objectives

4.1 Aim

The main aim of AetherGrid AI is to develop a reproducible urban dispatch simulation that combines dynamic routing, optimal vehicle-request assignment, multi-stop logistics optimization, and predictive repositioning of idle vehicles.

4.2 Objectives

The project has the following objectives:

1. Build a dynamic graph-based representation of an urban road network.
2. Model congestion, weather, hazards, and road closures as changing road conditions.
3. Generate emergency and logistics requests continuously using a controlled stochastic process.
4. Calculate efficient routes using Dynamic A*, Dijkstra, and HPA* where appropriate.
5. Build an ETA matrix between available vehicles and pending requests.
6. Use the Hungarian algorithm to solve the batch vehicle- request assignment problem.
7. Include deadline-aware penalties so that late emergency responses receive a higher assignment cost.
8. Use a Genetic Algorithm to solve multi-drop logistics problems with time windows.
9. Use Q-Learning or DQN to reposition only idle vehicles between dispatch batches.
10. Develop a real-time Mission Control dashboard for visualization and scenario control.
11. Maintain deterministic, seed-based simulation runs for testing and reproducibility.
12. Compare intelligent strategies against simple greedy baselines using measurable performance metrics.

5 Proposed Solution

AetherGrid is designed as a discrete-event simulation engine with a web-based visualization layer.

The proposed system contains the following major components.

5.1 Dynamic Road Network

The city is represented as a graph containing locations and roads. Every road has a current travel cost. The cost can change during the simulation as traffic, weather, or hazards change.

Road closures are handled differently from ordinary hazards. A hazard increases the cost of a road, whereas a closed road is removed from the graph entirely.

5.2 Pathfinding Suite

The project uses multiple pathfinding methods for different purposes:

- **Dynamic A*:** route execution using the current road costs.
- **Dijkstra:** calculation of shortest travel costs used for the vehicle-request ETA matrix.

- **HPA***: hierarchical pathfinding for larger or long-haul routes where a complete fine-grained search may be unnecessarily expensive.

5.3 Batch Dispatch

Requests are collected for a short dispatch interval. The proposed batch interval is:

$$\Delta = 5 \text{ s} \quad (1)$$

At each dispatch interval, the system calculates current travel costs, generates the ETA matrix, and solves the vehicle-request assignment problem.

5.4 Logistics Route Optimization

Emergency dispatch generally involves one immediate destination, while logistics operations may involve several destinations.

For multi-drop logistics missions, AetherGrid uses a Genetic Algorithm to search for routes that minimize travel cost while respecting delivery time windows.

5.5 Idle Vehicle Repositioning

The reinforcement-learning component is intentionally restricted to idle vehicles. It does not override a live dispatch.

Between batches, the RL policy may move an idle vehicle toward an area where future demand is expected to be higher. This allows the system to introduce a limited form of predictive behavior without making the safety-critical dispatch layer dependent on the convergence of a learning model.

6 System Workflow

The complete AetherGrid workflow is as follows:

1. The simulation initializes the road network, vehicles, requests, environmental conditions, and random seed.
2. Requests arrive according to the selected request generation model.
3. Requests are temporarily stored until the next dispatch batch.
4. The current road conditions are used to calculate the cost of each available road.
5. Roads affected by closures are removed from the graph.
6. Dijkstra or A* is used to calculate travel costs between vehicles and requests.
7. The resulting values form the vehicle-request cost matrix.
8. The Hungarian algorithm assigns vehicles to requests.

9. If a request belongs to a multi-drop logistics mission, the Genetic Algorithm based VRPTW solver is used instead.
10. Assigned vehicles execute their routes using Dynamic A*.
11. Idle vehicles that remain unassigned may be repositioned by the reinforcement-learning policy.
12. Vehicle states, routes, requests, and performance metrics are sent to the dashboard.
13. After a mission is completed, the vehicle returns to the idle pool and becomes available for the next batch.

This creates a continuous control loop in which the result of one dispatch cycle affects the state of the next cycle.

7 Mathematical Formulation

This section defines the mathematical models used by AetherGrid. Each equation is followed by a direct explanation of its variables and purpose.

7.1 Dynamic Road Network

The urban environment is represented as a weighted graph:

$$G_t = (V, E_t) \quad (2)$$

where:

- V is the set of locations or intersections.
- E_t is the set of roads available at time t .
- G_t represents the current road network.

The subscript t indicates that the graph can change during the simulation.

If a road is closed, it is removed from the current graph:

$$e \notin E_t \quad \text{if road } e \text{ is closed at time } t. \quad (3)$$

This prevents routing algorithms from selecting a closed road.

7.2 Dynamic Road-Cost Model

The travel cost of a road depends on its normal travel time, congestion, weather, and hazards.

The proposed cost function is:

$$C_e(t) = T_e (1 + \alpha \rho_e(t)) W_e(t) + \beta H_e(t) \quad (4)$$

where:

- $C_e(t)$ is the current travel cost of road e .
- T_e is the normal travel time of road e .
- $\rho_e(t)$ is the congestion level.
- α controls the effect of congestion.
- $W_e(t)$ is the weather multiplier.
- $H_e(t)$ represents the hazard level.
- β controls the hazard penalty.

A higher congestion or hazard level therefore increases the cost of using the affected road.

7.3 Request Arrival Model

Requests can be generated using a Poisson process. The probability of receiving k requests during a time interval t is:

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!} \quad (5)$$

where:

- $N(t)$ is the number of requests during the interval.
- k is the number of requests.
- λ is the average request arrival rate.
- t is the duration of the interval.

This allows the simulation to reproduce both ordinary demand and bursty demand.

7.4 A* Pathfinding

For every candidate node n , A* calculates:

$$f(n) = g(n) + h(n) \quad (6)$$

where:

- $g(n)$ is the actual cost from the starting node to n .
- $h(n)$ is the estimated cost from n to the destination.
- $f(n)$ is the estimated total cost through n .

The node with the lowest relevant evaluation value is considered first during the search.

The edge costs used by A* come from the dynamic road-cost model in Equation (4).

7.5 Estimated Time of Arrival

For vehicle i and request j , let P_{ij} denote the selected path.

The estimated travel time is:

$$ETA_{ij} = \sum_{e \in P_{ij}} C_e(t) \quad (7)$$

where:

- ETA_{ij} is the estimated travel time from vehicle i to request j .
- P_{ij} is the selected path.
- $C_e(t)$ is the current cost of road e .

The ETA values are used to construct the assignment matrix.

7.6 Vehicle-Request Cost Matrix

Suppose there are n available vehicles and m pending requests. The resulting matrix is:

$$\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix} \quad (8)$$

Here, C_{ij} represents the cost of assigning vehicle i to request j .

For a basic model:

$$C_{ij} = ETA_{ij} \quad (9)$$

The cost can later include additional penalties such as deadline violations.

7.7 Hungarian Assignment Model

Let x_{ij} be a binary decision variable:

$$x_{ij} = \begin{cases} 1, & \text{if vehicle } i \text{ is assigned to request } j, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

The objective is to minimize total assignment cost:

$$\min \sum_{i=1}^n \sum_{j=1}^m C_{ij} x_{ij} \quad (11)$$

Each vehicle can be assigned to at most one simultaneous request:

$$\sum_{j=1}^m x_{ij} \leq 1, \quad \forall i \quad (12)$$

Each request can be assigned to at most one vehicle:

$$\sum_{i=1}^n x_{ij} \leq 1, \quad \forall j \quad (13)$$

The Hungarian algorithm is used to solve this assignment problem efficiently.

7.8 Deadline-Aware Assignment

Let D_j be the required deadline of request j . If the current simulation time is t , the expected arrival time is:

$$A_{ij} = t + ETA_{ij} \quad (14)$$

The amount of lateness is:

$$L_{ij} = \max(0, A_{ij} - D_j) \quad (15)$$

If the vehicle arrives before the deadline, the lateness is zero.

The assignment cost can then be modified to:

$$C_{ij} = ETA_{ij} + \mu L_{ij} \quad (16)$$

where μ controls the importance of meeting the deadline.

This allows the assignment stage to consider both travel time and SLA compliance.

7.9 Vehicle Routing with Time Windows

For logistics missions involving multiple destinations, the project uses the Vehicle Routing Problem with Time Windows (VRPTW).

Let x_{ij} indicate whether a route travels directly from location i to location j .

The basic objective is:

$$\min \sum_{i \in V} \sum_{j \in V} d_{ij} x_{ij} \quad (17)$$

where d_{ij} is the travel cost between locations i and j .

Each location may also have an acceptable service window:

$$a_i \leq T_i \leq b_i \quad (18)$$

where:

- a_i is the earliest acceptable arrival time.
- b_i is the latest acceptable arrival time.
- T_i is the actual arrival time.

7.10 Genetic Algorithm Fitness

A candidate logistics route R is evaluated using a fitness cost:

$$F(R) = D(R) + \gamma P(R) \quad (19)$$

where:

- $D(R)$ is the travel cost of route R .
- $P(R)$ is the penalty caused by constraint violations.
- γ controls the importance of those violations.

The Genetic Algorithm attempts to minimize $F(R)$ through selection, crossover, and mutation.

7.11 Reinforcement-Learning Model

The idle-vehicle repositioning problem is represented as a Markov Decision Process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \quad (20)$$

where:

- $\mathcal{S}$ is the set of possible system states.
- $\mathcal{A}$ is the set of possible repositioning actions.
- P represents state-transition probabilities.
- R represents the reward function.
- γ is the discount factor.

The state can include information such as vehicle locations, vehicle availability, demand distribution, and road conditions.

7.12 Q-Learning Update

The Q-value is updated using:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (21)$$

where:

- s is the current state.
- a is the selected action.
- r is the reward received.
- s' is the next state.
- η is the learning rate.
- γ is the discount factor.
- a' represents a possible action in the next state.

The policy is trained to learn which repositioning actions are useful for future demand.

7.13 Repositioning Reward

The repositioning policy must balance faster future response with the additional cost of moving idle vehicles.

A simplified reward function is:

$$r = -w_T T - w_E E + w_S S \quad (22)$$

where:

- T represents response time.
- E represents additional energy consumption.
- S represents service performance.
- w_T is the response-time weight.
- w_E is the energy-consumption weight.
- w_S is the service-performance weight.

The reward encourages the system to reduce response time and improve service while avoiding unnecessary repositioning.

7.14 Deep Q-Network Extension

If the state space becomes too large for a Q-table, a Deep Q-Network can be used to approximate the Q-value:

$$Q(s, a; \theta) \quad (23)$$

where θ represents the trainable parameters of the neural network.

DQN is considered an extension of the initial Q-Learning implementation rather than a requirement for the basic simulation.

7.15 Vehicle Battery Model

For electric vehicles, the battery state is normalized between zero and one:

$$0 \leq B_i(t) \leq 1 \quad (24)$$

After movement, the battery state can be updated as:

$$B_i(t + \Delta t) = B_i(t) - E_i(\Delta t) \quad (25)$$

where:

- $B_i(t)$ is the battery level of vehicle i .
- $E_i(\Delta t)$ is the energy consumed during the interval.
- Δt is the simulation time interval.

A vehicle with insufficient battery should not be selected for a mission that it cannot complete safely.

8 System Architecture

AetherGrid follows a layered architecture so that the simulation engine, algorithms, and user interface remain separated.

8.1 Simulation Core

The simulation core contains:

- the road or grid network,
- vehicle agents,
- request agents,
- environmental conditions,
- vehicle battery state,
- the seeded request generator, and
- the discrete-event or tick-based simulation loop.

The core must be able to run without the web interface. This provides a headless mode for testing, benchmarking, and continuous integration.

8.2 Algorithm Layer

The algorithm layer contains independent strategy modules:

- Dynamic A*,

- Dijkstra,
- HPA*,
- Hungarian assignment,
- Genetic Algorithm based VRPTW,
- Q-Learning, and
- optional DQN.

Each strategy should have a clear interface so that different strategies can be compared without changing the simulation engine.

8.3 Web and Visualization Layer

The web layer provides Mission Control functionality.

The dashboard will display:

- the current road network,
- vehicle locations,
- active requests,
- assigned routes,
- road conditions,
- vehicle status,
- battery information,
- response-time statistics,
- SLA compliance, and
- simulation events.

The proposed interface uses WebSocket-based state updates. The backend can publish state changes at approximately 10 Hz while the browser renders the visual scene at up to 60 frames per second.

8.4 Scenario Laboratory

The dashboard will also contain a scenario laboratory through which the user can inject events such as:

- road closures,
- congestion,
- hazards,
- weather changes,

- demand bursts, and
- new emergency requests.

This makes it possible to demonstrate how the dispatch strategies react to controlled disruptions.

9 Proposed Technology Stack

The following technology stack is proposed for implementation.

Layer	Proposed Technology / Approach
Programming Language	Python 3.11
Numerical Computation	NumPy and standard Python libraries
Graph Algorithms	Custom implementations of A*, Dijkstra, and HPA*
Assignment	Hungarian algorithm implementation
Route Optimization	Genetic Algorithm for VRPTW
Reinforcement Learning	Q-Learning with optional DQN extension
Simulation	Seeded discrete-event / tick-based simulation engine
Backend	Python web backend with WebSocket communication
Frontend	HTML, CSS, JavaScript, and Canvas-based visualization
Testing	Unit tests and property-based tests
Version Control	Git and GitHub
Continuous Integration	Automated seed-stable regression and invariant tests

Table 1: Proposed technology stack for AetherGrid AI.

The project intentionally uses established algorithms and standard software components. The primary engineering challenge is their correct integration into one reproducible system.

10 Implementation Methodology

The project will be implemented incrementally so that each stage produces a working and testable result.

10.1 Phase 1: Simulation Foundation

The first phase will implement:

- the road graph,
- vehicle and request models,
- environmental conditions,
- the battery state model,
- the seeded request generator, and
- the simulation loop.

Exit criterion: a seed-stable headless simulation run.

10.2 Phase 2: Routing and Assignment

The second phase will implement:

- Dynamic A*,
- Dijkstra,
- HPA*,
- ETA matrix generation,
- Hungarian assignment,
- deadline-aware costs, and
- baseline greedy dispatch.

Exit criterion: routing and assignment tests pass against known small examples and baseline strategies.

10.3 Phase 3: Logistics and Reinforcement Learning

The third phase will implement:

- VRPTW representation,
- Genetic Algorithm route optimization,
- Q-Learning for idle-vehicle repositioning, and
- optional DQN extension.

Exit criterion: each strategy can be run independently and compared under identical seeded scenarios.

10.4 Phase 4: Mission Control Dashboard

The fourth phase will implement:

- live map visualization,

- vehicle telemetry,
- request monitoring,
- route visualization,
- scenario injection,
- strategy switching, and
- performance charts.

Exit criterion: a complete scenario can be run from the dashboard while the dispatch strategy is changed during operation.

10.5 Phase 5: Testing and Evaluation

The final phase will focus on:

- performance benchmarking,
- reproducibility,
- stress testing,
- disruption testing,
- baseline comparison, and
- final documentation.

Exit criterion: all predefined evaluation experiments have been executed and documented.

11 Testing and Validation

AetherGrid will use both functional tests and performance tests.

11.1 Functional Invariants

Important system properties include:

- No route may traverse a closed road.
- A vehicle cannot be assigned to two simultaneous requests.
- A request cannot be assigned to two vehicles.
- Battery levels remain within their valid range.
- Completed vehicles return to the available pool.
- The same random seed produces the same simulation trace.

11.2 Assignment Validation

For small problem sizes, the Hungarian solution can be compared against brute-force enumeration. For larger problem sizes, it can be compared against a greedy nearest-vehicle baseline.

The expected property is that the optimized assignment should not have a higher total assignment cost than the corresponding greedy assignment under the same cost matrix.

11.3 Routing Validation

A* and Dijkstra will be tested on small graphs for which the optimal path cost can be calculated independently.

The tests will also verify that introducing a road closure changes the route when the original route is no longer valid.

11.4 Reinforcement-Learning Validation

The RL component will be evaluated by comparing:

1. no repositioning,
2. random repositioning,
3. heuristic repositioning, and
4. learned repositioning.

The comparison will use identical demand scenarios and random seeds.

12 Evaluation Metrics

The following metrics will be used to compare the proposed strategies.

12.1 Average Response Time

The average response time is:

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i \quad (26)$$

where N is the number of completed requests and T_i is the response time of request i .

A lower value indicates faster average response.

12.2 SLA Compliance Rate

The percentage of requests completed within their deadlines is:

$$SLA_{\text{rate}} = \frac{N_{\text{on-time}}}{N_{\text{total}}} \times 100 \quad (27)$$

where:

- $N_{\text{on-time}}$ is the number of requests completed within their deadlines.
- N_{total} is the total number of requests.

A higher value represents better deadline compliance.

12.3 Average Travel Distance

The average distance travelled per mission is:

$$\overline{D} = \frac{1}{N} \sum_{i=1}^N D_i \quad (28)$$

where D_i is the distance travelled during mission i .

12.4 Average Energy Consumption

The average energy consumption can be measured as:

$$\overline{E} = \frac{1}{N} \sum_{i=1}^N E_i \quad (29)$$

where E_i is the energy consumed by mission i .

12.5 Algorithm Runtime

The runtime of a decision-making algorithm is:

$$T_{\text{runtime}} = T_{\text{finish}} - T_{\text{start}} \quad (30)$$

The measured runtime will be compared with the five-second batch interval defined in Equation (1).

13 Overall Optimization Objective

AetherGrid attempts to balance response time, service quality, energy consumption, and travel distance.

A simplified multi-objective cost is:

$$J = w_1 \overline{T} + w_2 (1 - SLA_{\text{rate}}) + w_3 \overline{E} + w_4 \overline{D} \quad (31)$$

where:

- $\overline{T}$ is the average response time.
- SLA_{rate} is the SLA compliance rate.

- $\bar{E}$ is the average energy consumption.
- $\bar{D}$ is the average travel distance.
- w_1, w_2, w_3, w_4 are weights representing the importance of the corresponding objectives.

The system seeks to minimize J while satisfying the operational constraints of the simulation.

The exact weights will be selected during experimentation rather than being presented as predetermined results.

14 Experimental Design

The evaluation will use controlled and reproducible scenarios.

14.1 Baseline Strategies

The intelligent strategies will be compared with simpler methods:

- **Nearest Vehicle:** assigns the closest available vehicle to each request.
- **First-Come-First-Served:** processes requests in arrival order.
- **Greedy Assignment:** repeatedly selects the currently cheapest vehicle-request pair.
- **Hungarian Assignment:** performs batch-level global assignment.
- **Hungarian + RL:** combines optimized dispatch with learned idle-vehicle repositioning.

14.2 Scenario Categories

The following scenarios will be tested:

1. Normal traffic and normal request volume.
2. High traffic.
3. Bursty emergency demand.
4. Multiple simultaneous requests.
5. Road closure during an active simulation.
6. Hazard introduced during an active route.
7. Uneven spatial demand.
8. High vehicle utilization.
9. Mixed emergency and logistics requests.

Each strategy will be evaluated on identical seeded scenarios to ensure that the comparison is fair.

15 Evaluation Hypotheses

The project will use measurable hypotheses rather than claiming performance improvements before the experiments are completed.

15.1 Hypothesis 1: Batch Matching

Under bursty demand, the hypothesis is that global batch assignment will perform better than a purely greedy nearest-vehicle strategy.

The comparison will use identical request streams and vehicle states.

The hypothesis will be considered unsupported if the measured difference is negligible across the selected scenarios.

15.2 Hypothesis 2: Predictive Repositioning

The second hypothesis is that repositioning idle vehicles toward anticipated demand can reduce future response time compared with dispatch without repositioning.

The additional energy and movement cost will also be measured. An improvement in response time alone will not be considered sufficient if the additional operational cost becomes excessive.

15.3 Hypothesis 3: Disruption Handling

The third hypothesis is that the dynamic cost field will allow the system to respond to hazards and road closures without violating routing constraints.

The test will verify that:

- affected routes are recalculated,
- closed roads are not traversed,
- vehicles continue operating when alternative routes exist,
- no invalid vehicle assignment is produced.

16 Reproducibility and Continuous Integration

Reproducibility is an important design requirement.

Every simulation run will use an explicit random seed. When the same configuration and seed are used, the generated request sequence and simulation behavior should be reproducible.

Automated tests will be executed whenever changes are made to the project.

Important automated checks include:

- no closed-edge traversal,
- valid battery bounds,

- valid vehicle-request assignments,
- deterministic seeded runs,
- assignment-cost comparison against baseline methods,
- route validity, and
- regression testing of core simulation behavior.

This approach allows the simulation engine to operate in a headless mode for automated testing while the same engine can drive the live dashboard for demonstrations.

17 Scalability

The architecture is designed so that the simulation engine is separated from the visualization layer.

The assignment problem can be solved using the Hungarian algorithm with cubic time complexity:

$$O(n^3) \tag{32}$$

for an $n \times n$ assignment matrix.

This makes the assignment stage practical for moderate fleet sizes within the fixed dispatch interval. For larger graphs, HPA* can be used to reduce the routing search space.

The separation between the simulation engine and dashboard also ensures that rendering workload does not directly determine the correctness of the simulation.

18 Project Scope

18.1 Included in Scope

The project will include:

- dynamic road-network simulation,
- heterogeneous vehicle simulation,
- emergency and logistics requests,
- congestion and environmental effects,
- road closures and hazards,
- Dynamic A*,
- Dijkstra,
- HPA*,
- Hungarian assignment,

- Genetic Algorithm based VRPTW,
- Q-Learning,
- optional DQN,
- vehicle battery modeling,
- real-time dashboard,
- scenario laboratory,
- reproducible experiments, and
- baseline performance comparison.

18.2 Outside the Initial Scope

The following are not required for the first complete version:

- deployment on a real city-wide transportation network,
- direct control of real vehicles,
- dependence on proprietary real-time traffic data,
- production-grade autonomous driving,
- guaranteed real-world emergency response performance.

The system is a simulation and engineering demonstration. Its experimental results should therefore be interpreted within the conditions of the implemented simulation.

19 Risks and Mitigation

19.1 Scope Creep

The project combines several algorithm families. To control the scope, every algorithm will be implemented behind a fixed interface and developed in phases.

19.2 Real-Time Performance

If routing becomes expensive on larger graphs, HPA* can be introduced for long-haul searches. Assignment runtime will also be measured against the five-second batch interval from the beginning of development.

19.3 Reinforcement-Learning Instability

The RL component will remain advisory and will only control idle vehicle repositioning. It will never override an active dispatch. Therefore, an incompletely trained policy cannot invalidate an emergency assignment.

19.4 Evaluation Ambiguity

The experiments will be defined before final benchmarking. All strategies will be evaluated using identical seeds, scenarios, and metrics.

20 Expected Outcomes

At the completion of the project, the following outcomes are expected:

1. A working seeded urban dispatch simulation engine.
2. A dynamic road graph capable of representing congestion, weather, hazards, and road closures.
3. A routing system supporting Dynamic A*, Dijkstra, and HPA*.
4. A batch assignment system using the Hungarian algorithm.
5. A logistics routing module based on Genetic Algorithm and VRPTW.
6. A reinforcement-learning module for idle-vehicle repositioning.
7. A real-time Mission Control dashboard.
8. A scenario laboratory for controlled disruptions.
9. Automated tests for important system invariants.
10. Reproducible benchmark results comparing multiple dispatch strategies.

The final result will therefore be a complete software system rather than an isolated implementation of a single algorithm.

21 Project Deliverables

The planned deliverables are:

22 Conclusion

AetherGrid AI proposes a practical simulation-based approach to dynamic urban dispatch. Instead of treating routing, assignment, and future vehicle positioning as one problem, the system separates them into cooperating layers.

The dynamic road model represents changing travel conditions. The pathfinding layer calculates routes using current road costs. The Hungarian algorithm provides global batch-level assignment for one-to-one dispatch. The Genetic Algorithm handles multi-stop logistics routing with time windows. Finally, the reinforcement-learning layer provides limited predictive behavior by repositioning idle vehicles between dispatch cycles.

The project is intentionally designed around established algorithms rather than claiming algorithmic novelty. Its contribution is the engineering integration of these methods into

Deliverable	Description
Simulation Engine	Seeded urban road, vehicle, request, and environment simulator.
Routing Module	Dynamic A*, Dijkstra, and HPA* implementations.
Assignment Module	Hungarian vehicle-request matching with deadline-aware costs.
Logistics Module	Genetic Algorithm based VRPTW route optimization.
RL Module	Q-Learning based idle-vehicle repositioning with optional DQN extension.
Mission Control	Real-time browser dashboard for visualization and monitoring.
Scenario Lab	Controlled injection of traffic, hazards, closures, and demand bursts.
Testing Suite	Unit, property, invariant, and reproducibility tests.
Benchmark Suite	Seeded comparison against greedy and other baseline strategies.
Documentation	Technical report, architecture documentation, and usage instructions.

Table 2: Planned AetherGrid deliverables.

a reproducible, testable, real-time simulation environment.

The final system will allow different strategies to be tested under the same controlled scenarios. This makes it possible to determine, using measurable evidence, whether global assignment and predictive idle-vehicle repositioning actually improve response performance, service compliance, and operational efficiency.